COMP 511

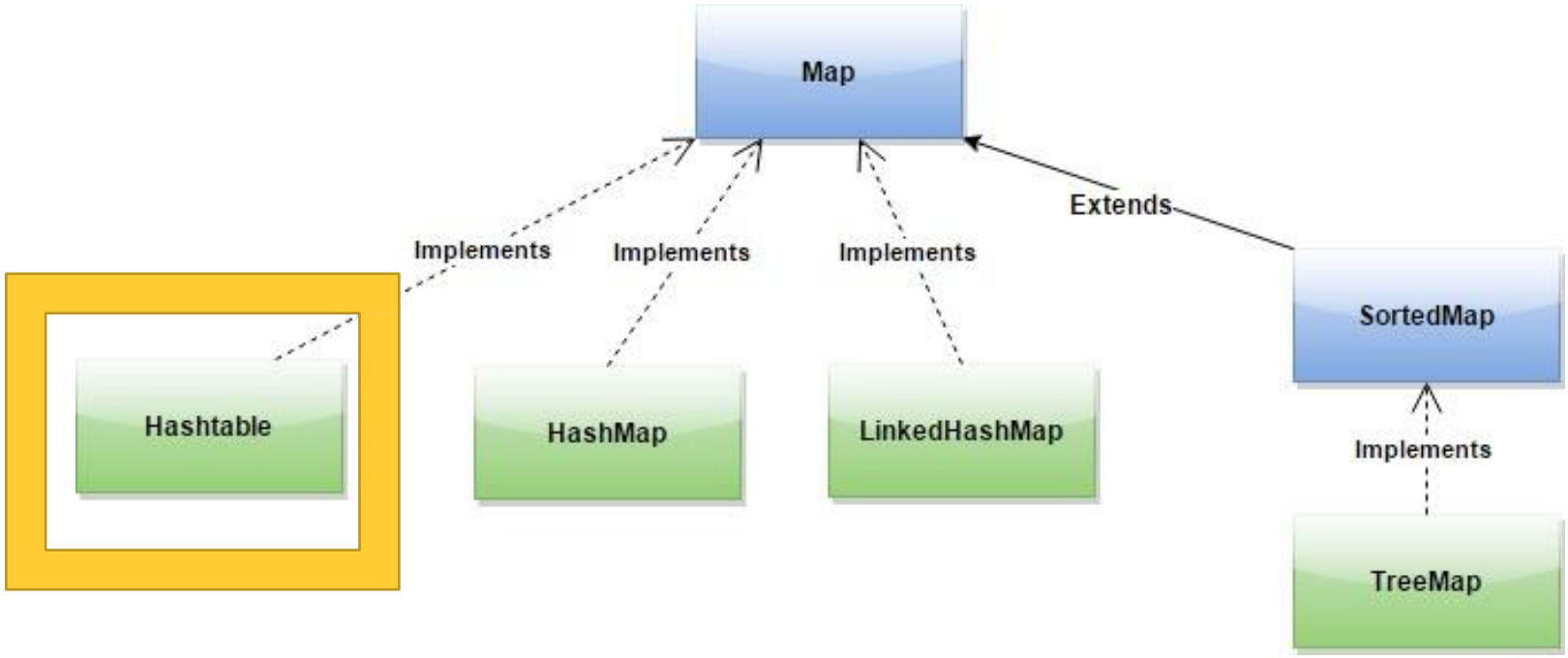
# Computer Programming Foundations II

## Hash Table

# The Map Interface

- The Map interface maps unique keys to values.

- A map cannot contain duplicate keys, and each key can map to at most one value



# Hash Table

- Hash table is a *key* based data structure, which allows a decrease in time complexity
- A *hash function* takes a key as input and provides an array index into which we store a reference to the key's associated value.

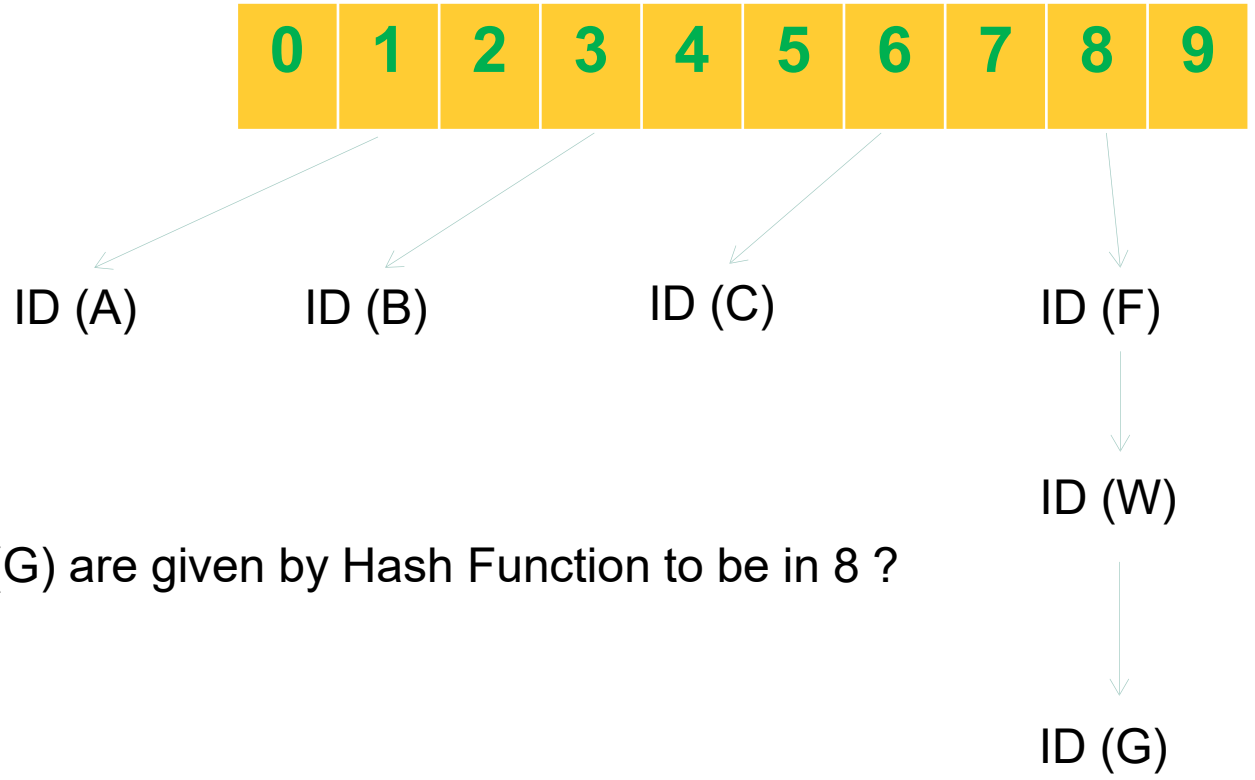
- Key = INFS => Value : INFS' ID Hash(INFS) -> 5
- Key = SWE => Value : SWE' ID Hash(SWE) -> 8

What if Hash (CS) -> 8 ? **Collision**

# Collision

- There are many ways to handle collision occurrences within hash table implementations.

## Chaining




ID (W) and ID (G) are given by Hash Function to be in 8 ?

# Hash Function

- Hash function should minimize the collision .

- Chaining should be evenly distributed.

## Examples

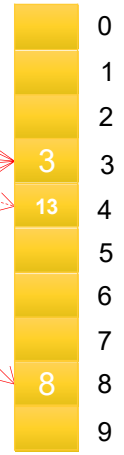
- Prime Number e.g. : Hash(x) = X % 7

# Linear Probing

In Closed Hashing (also referred to as *Open Addressing*), all elements are stored *within* the hash table itself. A dataset involving N key/value pairs requires a hash table (array) of size greater than or equal to N.

h(x) = x % size
**h(x) = [x + f(i)] % size**
f(i) = i where i=0,1,2,…

Key space:
8
3
13
6
4
10




h(13) = [13 + f(0)] % 10 = 3
If there is a collision, we take the next value of i.
[13 + f(1)] % 10 = 4
Index 4 is available, so there is no collision. Had index 4 been occupied (i.e. another collision), we simply move on to index 5, and so on… .

Let's proceed with key:6

# Linear Probing and Clustering

To retrieve key: 4, we process 4 using the hash function with i=0. Since 4 isn't present at that location, we increment i to 1, and so on…

Let's say we wish to search for (non-existent) key 24.

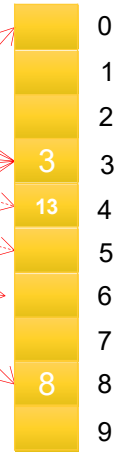
24%10 = 4 (not present)

5 (not present)

6 empty -> **key not found**.

h(x) = x % size
**h(x) = [x + f(i)] % size**
f(i) = i where i=0,1,2,…

Key space:
8
3
13
6
4
10




Is there a problem with this approach? *Clustering*

Clustering indicates that our hash function is not distributing the data evenly throughout the hash table.

Note: time complexity is still > O(1)

# Linear Probing

Another example:

**h(x) = [x + f(i)] % size**
f(i) = i where i=0,1,2,…

Hash(89) = 9
Hash(18) = 8
Hash(49) = 9
Hash(58) = 8
Hash(9) = 9

| | After insert 89 | After insert 18 | After insert 49 | After insert 58 | After insert 9 |
|---|---|---|---|---|---|
| 0 | | | 49 | 49 | 49 |
| 1 | | | | 58 | 58 |
| 2 | | | | | 9 |
| 3 | | | | | |
| 4 | | | | | |
| 5 | | | | | |
| 6 | | | | | |
| 7 | | | | | |
| 8 | | 18 | 18 | 18 | 18 |
| 9 | 89 | 89 | 89 | 89 | 89 |

The *search* algorithm follows the same probe sequence as the insert algorithm.
- Search for 58 involves 4 probes.
- How many probes would a search for 19 require?

Again, we see the **Clustering** problem. As long as our table is large enough, we can always find a free cell. However the time to do so may become quite large

# Linear Probing Exercise

*Exercise: note that size is now 11.*

**h(x) = [x + f(i)] % size**
f(i) = i where i=0,1,2,…

Hash(20)
Hash(30)
Hash(2)
Hash(13)
Hash(25)
Hash(24)
Hash(10)
Hash(9)

| | |
|---|---|
| 0 | |
| 1 | |
| 2 | |
| 3 | |
| 4 | |
| 5 | |
| 6 | |
| 7 | |
| 8 | |
| 9 | |
| 10 | |

# Linear Probing Ex. Solution

**h(x) = [x + f(i)] % size**
f(i) = i where i=0,1,2,…

Hash(20):9
Hash(30):8
Hash(2):2
Hash(13):2,3
Hash(25):3,4
Hash(24):2,3,4,5
Hash(10):10
Hash(9):9,10,0

| | |
|---|---|
| 0 | 9 |
| 1 | |
| 2 | 2 |
| 3 | 13 |
| 4 | 25 |
| 5 | 24 |
| 6 | |
| 7 | |
| 8 | 30 |
| 9 | 20 |
| 10 | 10 |

Average Probe:
(1+1+1+2+2+4+1+3)/8 = 15/8

To reduce the clustering problem, we will now
Discuss Quadratic Probing.

# Quadratic Probing

h(x) = [x + f(i)] % size
**f(i) = $i^2$ where**
**i=0,1,2,…**

Key space:
8
3
13
23
43
10

h(23) = [23 + f(0)] % 10 = 3 (already taken)
h(23) = [23 + f(1)] % 10 = 4 (already taken)
**h(23) = [23 + f(2)] % 10 = 7** (available)
**= (23 + 4) % 10 = 7**

Note that this reduces clustering, and thus improves search time.

# Quadratic Probing Exercise

h(x) = [x + f(i)] % size
**f(i) = $i^2$ where i=0,1,2,…**

*Exercise*
Key space:
872
125
110
847
356
445

| | |
|---|---|
| 110 | 0 |
| | 1 |
| 872 | 2 |
| | 3 |
| | 4 |
| **125** | 5 |
| 356 | 6 |
| **847** | 7 |
| | 8 |
| 445 | 9 |

# Double Hashing

**Linear Probing:**
$h(x) = [x + f(i)] \% \text{ size}$
$f(i) = i$ where $i=0,1,2,\ldots$

**Quadratic Probing**
$h(x) = [x + f(i)] \% \text{ size}$
$f(i) = i^2$ where $i=0,1,2,\ldots$

**Double Hashing**
$h(x) = [x + f(i)] \% \text{ size}$
$f(i) = i * h_2(x)$ where $i=0,1,2,\ldots$
(We must be careful not to allow $h_2(x)$ to equal zero.)

An often used hash function for $h_2(x)$ is:
$r - (x \% r)$ $\quad r \in \text{Prime}$ $\quad r < \text{Table size}$

*Example:*
$h(x) = [x + f(i)] \% 10$
$f(i) = i * h_2(x)$ where $i=0,1,2,\ldots$
$h_2(x) = 7 - (x \% 7)$
note: 7 is the largest prime number < table size

| | 89 | 18 | 49 | 58 | 69 |
|---|---|---|---|---|---|
| 0 | | | | | |
| 1 | | | | | |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |
| 5 | | | | | |
| 6 | | | | | |
| 7 | | | | | |
| 8 | | 18 | | | |
| 9 | 89 | 89 | | | |

# Double Hashing

**Linear Probing:**
h(x) = [x + f(i)] % size
f(i) = i where i=0,1,2,…

**Quadratic Probing**
h(x) = [x + f(i)] % size
f(i) = $i^2$ where i=0,1,2,…

**Double Hashing**
h(x) = [x + f(i)] % size
f(i) = i * $h_2(x)$ where i=0,1,2,…
(We must be careful not to allow $h_2(x)$ to equal zero.)

An often used hash function for $h_2(x)$ is:
r – (x % r)  r∈Prime  r<Table size

*Example:*
h(x) = [x + f(i)] % 10
f(i) = i * $h_2(x)$ where i=0,1,2,…
$h_2(x)$ = 7 – (x % 7)
note: 7 is the largest prime number < table size

| | 89 | 18 | 49 | 58 | 69 |
|---|---|---|---|---|---|
| 0 | | | | | |
| 1 | | | | | |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |
| 5 | | | | | |
| 6 | | | 49 | | |
| 7 | | | | | |
| 8 | | 18 | 18 | | |
| 9 | 89 | 89 | 89 | | |

| x | i | |
|---|---|---|
| 49 | 0 | => 49 % 10 = 9 taken |
| | 1 | => 49 + (1 * (7 – (49 % 7)) |
| | | (1 * (7 - 0)) |
| | | 56 % 10 = 6 |

# Double Hashing

**Linear Probing:**
h(x) = [x + f(i)] % size
f(i) = i where i=0,1,2,…

**Quadratic Probing**
h(x) = [x + f(i)] % size
f(i) = $i^2$ where i=0,1,2,…

**Double Hashing**
h(x) = [x + f(i)] % size
f(i) = i * $h_2(x)$ where i=0,1,2,…
(We must be careful not to allow $h_2(x)$ to equal zero.)

An often used hash function for $h_2(x)$ is:
r – (x % r)  r∈Prime  r<Table size

*Example:*
h(x) = [x + f(i)] % 10
f(i) = i * $h_2(x)$ where i=0,1,2,…
$h_2(x)$ = 7 – (x % 7)
note: 7 is the largest prime number < table size

| | 89 | 18 | 49 | 58 | 69 |
|---|---|---|---|---|---|
| 0 | | | | | |
| 1 | | | | | |
| 2 | | | | | |
| 3 | | | | 58 | |
| 4 | | | | | |
| 5 | | | | | |
| 6 | | | 49 | 49 | |
| 7 | | | | | |
| 8 | | 18 | 18 | 18 | |
| 9 | 89 | 89 | 89 | 89 | |

| x | i | |
|---|---|---|
| 58 | 0 | => 8 taken |
| | 1 | => 58 + (1 * (7 – (58 % 7))) |
| | | 7 - 2 |
| | | 58 + 5 = 63 |
| | | 63 % 10 = 3 |

# Double Hashing

**Linear Probing:**
h(x) = [x + f(i)] % size
f(i) = i where i=0,1,2,…

**Quadratic Probing**
h(x) = [x + f(i)] % size
f(i) = $i^2$ where i=0,1,2,…

**Double Hashing**
h(x) = [x + f(i)] % size
f(i) = i * $h_2(x)$ where i=0,1,2,…
(We must be careful not to allow $h_2(x)$ to equal zero.)

An often used hash function for $h_2(x)$ is:
r – (x % r)  r∈Prime  r<Table size

| x | i | |
|---|---|---|
| 69 | 0 | => 9 taken |
| | 1 | => 69 + (1 * (7 – (69 % 7))) |
| | | 7 - 6 |
| | | 69 + 1 = 70 |
| | | 70 % 10 = 0 |

*Example:*
h(x) = [x + f(i)] % 10
f(i) = i * $h_2(x)$ where i=0,1,2,…
$h_2(x)$ = 7 – (x % 7)
note: 7 is the largest prime number < table size

| | 89 | 18 | 49 | 58 | 69 |
|---|---|---|---|---|---|
| 0 | | | | | 69 |
| 1 | | | | | |
| 2 | | | | | |
| 3 | | | | 58 | 58 |
| 4 | | | | | |
| 5 | | | | | |
| 6 | | | 49 | 49 | 49 |
| 7 | | | | | |
| 8 | | 18 | 18 | 18 | 18 |
| 9 | 89 | 89 | 89 | 89 | 89 |

# Hash Table Exercises

Insert the following keys into the given hash table using linear probing as collision resolution technique.

keys: 12, 18, 13, 2, 3, 23, 5 and 15 hash function h(k) = k mod 10

| | |
|---|---|
| 0 | |
| 1 | |
| 2 | |
| 3 | |
| 4 | |
| 5 | |
| 6 | |
| 7 | |
| 8 | |
| 9 | |

| | |
|---|---|
| 0 | |
| 1 | |
| 2 | 12 |
| 3 | 13 |
| 4 | 2 |
| 5 | 3 |
| 6 | 23 |
| 7 | 5 |
| 8 | 18 |
| 9 | 15 |